Operators

CSL 100/MAL400

This Class

- All about operators
- Related reading: Chapter 3

Operators

- Operators are the building blocks of expressions
- Have their own syntax
- We will learn this syntax the next few lectures

Types of operators

- Arithmetic
- Unary
- Relational and logical
- Assignment
- Conditional

Arithmetic Operators

- Operate on int, float, double (and char)

Op	Meaning	Example	Remarks
+	Addition	9+2 is 11	
		9.1+2.0 is 11.1	
-	Subtraction	9-2 is 7	
		9.1-2.0 is 7.1	
*	Multiplication	9*2 is 18	
		9.1*2.0 is 18.2	
/	Division	9/2 is 4	Integer division
		9.1/2.0 is 4.55	Real division
%	Remainder	9%2 is 1	Only for int

The / operator

- When both (left and right) operand of / are of type int
  - The result is the integral part of the real division
  - The result is of type int
- Examples

  9/4 is 2

  1/2 is 0

The / operator

- When at least one (left or right or both) operands of / are of type float (double)
  - The result is the real division
  - The result is of type float (double)
- Examples

  9/4.0 is 2.250000

  1.0/2 is 0.500000,

  so is 1/2.0

  and 1.0/2.0

The % operator

- The remainder operator % returns the integer remainder of the result of dividing its first operand by its second.
- Both operands must be integers.
- Defined only for integers (int and long)

  4%2 is 0

  31%4 is 3

Divison(/) and Remainder(%)

- Second argument can not be 0
  - Run time error
- For integers a and b (b≠0), / and % have the following relation

$$a = (a/b)*b + (a\%b)$$

- If a or b or both are negative, the result of / and % is system dependent.
  - But satisfies the above relation

Quirks of C Arithmetic

- Most of the arithmetic operators follow the rules of regular arithmetic
- Exceptions can be annoying
  - Modulus operation troublesome for negative operands
    - Usually gives the remainder the sign of the first operand
    - -11 % 4 = -3

Implicit Type Casting in C Arithmetic

- General rule for mixed type arithmetic
  - Final result will have highest possible precision consistent with operand types
  - Small boxes → big boxes
- Examples
  - *float + double = double*
  - *float + long double = long double*
  - *int + float = float*
  - *char + float = float*
  - *…*

Explicit Type Casting in C Arithmetic

- Can cast an arithmetic expression to a specific data type
  - Syntax *(data type) expression;*
  - *E.g. ((int) f) % 2*

Precedence and Associativity

- Precedence → which operations to carry out first

- Associativity → which operations with the same precedence to carry out first
  - English uses left→right
  - Urdu uses right→left
  - C uses both, in different cases

Precedence and Associativity for Arithmetic Expressions

Precedence	Operators
High	* / %
Low	+ -

e.g. a – b / c * d = a – ((b/c)*d)

Like in regular arithmetic, can use brackets to clarify correct order of operations, e.g. (a-b)/(c*d)

Associativity goes left → right for arithmetic expressions

Types of operators

- Arithmetic
- **Unary**
- Relational and logical
- Assignment
- Conditional

Unary operators

Operator	Description
-	Negative of an expression
++/--	Increment/decrement a variable
sizeof	Output memory box size for a variable
type	Type-casting

Unary Operators - Negative

- Operators that take only one argument (or operand)
  - -5
  - -b
- Observe that – is both an arithmetic and unary operator
  - Meaning depends on context
  - This is called overloading

Unary Operators – Increment and Decrement

- Increment (++) increases a variable by 1
- Decrement (--) – decreases a variable by 1
- Both work on all variable types
- Syntax
  - *++variable* is the pre-increment operator
    - Means increment, then use
  - *variable++* is the post-increment operator
    - Means use, then increment

```
int main(){
        char a = 'A';     float b = 3.31;
        printf("%c\t%f\n",++a,b++);
       printf("%c\t%f",--a,b--);
      return 0;
}
```

```
B       3.31
A       4.31
```

Unary operators - sizeof

- Syntax
  - *sizeof var*
  - *sizeof(type)*
- Returns size of the operand in bytes
  - sizeof(char) will return 1
  - sizeof(float) will (mostly) return 4
- Very useful when you are porting programs across computers

Unary Operators - typecast

- Syntax
  - (type) var
- We have already seen this
- What will be the output of this program?

```
int main(){
        double a = 67.2;
        printf("size is %d\n", sizeof a);
        printf("size is %d\n", sizeof((char) a));
        printf("%c", (char) a);
       return 0;
}
```

Size is 8
Size is 1
C

Precedence

- Above arithmetic operators, only below brackets

- If a is 1 and b is 2, what will a + -b be evaluated as?
  - -1

Associativity

- From right to left
  - Important to remember this
  - Most other operators' associativity is left to right
- What will this program print?

```
int main(){
        int a = 1;
        printf("%d", - ++a);
       return 0;
}
```

-2

Types of operators

- Arithmetic
- Unary
- **Relational and logical**
- Assignment
- Conditional

Relational Operators



- Compare two quantities

Operator	Function
>	Strictly greater than
>=	Greater than or equal to
<	Strictly less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

- Work on int, char, float, double…

Examples

Rel. Expr.	Result	Remark
3>2	1	
3>3	0	
‘z’ > ‘a’	1	ASCII values used for char
2 == 3	0	
‘A’ <= 65	1	'A' has ASCII value 65
‘A’ == ‘a’	0	Different ASCII values
(‘a’ – 32) == ‘A’	1	
5 != 10	1	
1.0 == 1	AVOID	May give unexpected result due to approximation

Avoid mixing int and float values while comparing. Comparison with floats is not exact!

Example

- Problem: Input 3 positive integers. Print the count of inputs that are even and odd.
  - Do not use if-then-else

INPUT
10
5
3

OUTPUT
Even=1
Odd=2

```
int a; int b; int c;
int cEven; // count of even i
scanf("%d%d%d", &a,&b,&c); // input a,b,c

// (x%2 == 0) evaluates to 1 if x is Even,
//                          0 if x is Odd
cEven = (a%2 == 0) + (b%2 == 0) + (c%2 == 0);
printf("Even=%d\nOdd=%d", cEven, 3-cEven);
```

Logical Operators

Logical Op	Function	Allowed Types
&&	Logical AND	char, int, float, double
||	Logical OR	char, int, float, double
!	Logical NOT	char, int, float, double

Remember

- value 0 represents false.
- any other value represents true. Compiler returns 1 by default

Truth Tables

E1	E2	E1 && E2	E1 || E2
0	0	0	0
0	Non-0	0	1
Non-0	0	0	1
Non-0	Non-0	1	1

E	!E
0	1
Non-0	0

Examples

Expr	Result	Remark
2 && 3	1	
2 || 0	1	
‘A’ && ‘0’	1	ASCII value of ‘0’≠0
‘A’ && 0	0	
‘A’ && ‘b’	1	
! 0.0	1	0.0 == 0 is **guaranteed**
! 10.05	0	Any real ≠ 0.0
(2<5) && (6>5)	1	Compound expr

Precedence and associativity

- Not has same precedence as equality operator
- And and Or are lower than relational operators
- Or has lower precedence than And
- Associativity goes left to right
- 2 == 2 && 3 == 1 || 1==1 || 5==4 is true

Operator Precedence

HIGH



LOW

Operators	Description	Associativity
(unary) + -	Unary plus/minus	Right to left
* / %	Multiply, divide, remainder	Left to right
+ -	Add, subtract	Left to right
< > >= <=	Relational operators	Left to right
== !=	Equal, not equal	Left to right
&&	And	Left to right
||	Or	Left to right
=	Assignment	Right to left

Class Quiz

- What is the value of expression:

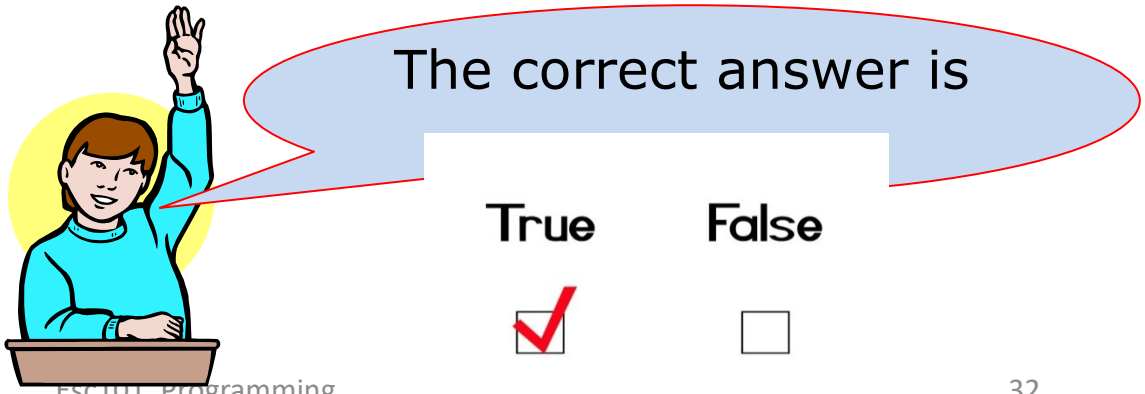
```
0 <= 10 <= 4
```

a) Compile time error

b) Run time crash

c) False (0)

d) True (1)



Next Class

- Related reading in parentheses
  - Conditional operators (3.5)
  - Library functions (3.6)
  - Using expressions in programs (Ch 5)